COS Open-Book Notes (Lectures 10–20)

Ashoka University COS

April 23, 2026

Contents

How to Use This Document to Pass the Exam **5**

Diagram Placeholders **5**

1 Floating Point and Computer Arithmetic (Lectures 10–11) **6**

1.1 IEEE 754 Formats and Fields (L10, L11) 6
1.2 Exponent Bias, Encoding, and Special Exponents (L10, L11) 6
1.3 Normalized, Denormal, Zero, Infinity, and NaN (L10) 7
1.4 Encoding and Decoding Procedure (L11) 7
1.5 Floating-Point Addition and Subtraction Algorithm (L10, L11) 8
1.6 Floating-Point Multiplication Algorithm (L10) 8
1.7 Guard, Round, and Sticky Bits (L10) 9
1.8 Floating-Point Division Overview (L11) 9
1.9 Goldschmidt Division Method (L11) 10
1.10 Newton-Raphson Division Method (L11) 10
1.11 FP Hardware: Multi-Cycle and Pipelined Units (L10, L11) 11
1.12 MIPS FPU Architecture (L10) 11
1.13 MIPS FP Instructions and Data Movement (L10) 12
1.14 FP Syscalls and Example (L10) 12
1.15 Integer Multiplication Complexity (L10) 13

2 Integer Arithmetic and ALU (Lectures 11–12) **13**

2.1 Unsigned Shift-Add Multiplier (L12) 13
2.2 Signed Multiplication and Sign Handling (L12) 14
2.3 Booth's Encoding and Algorithm (L12) 14

2.4 Full Adder, Ripple-Carry Adder, and 2's Complement Subtraction (L11, L12) 15
2.5 Carry Lookahead Adder (CLA) (L11) 15
2.6 Hybrid CLA/RCA Adders (L11) 16
2.7 32-bit MIPS ALU Design (L12) 16
2.8 Overflow Detection and SLT Support (L12) 16
2.9 MIPS Instruction Execution Cycle (L12) 17
2.10 Combinational vs Sequential Elements and Register File (L12) 17

3 Processor Design: Single-Cycle MIPS (Lectures 13–14) 18

3.1 ISA-Driven Datapath Design and Building Blocks (L13) 18
3.2 R-Type Datapath (L13) 18
3.3 Load Datapath (L13) 19
3.4 Store Datapath (L13) 19
3.5 Branch Datapath (L13) 20
3.6 Combining Datapaths with Multiplexers (L13) 20
3.7 Control Signals for Single-Cycle MIPS (L14) 21
3.8 ALU Control Logic (L14) 21
3.9 Main Control Unit Design (L14) 22
3.10 Jump Instruction Handling (L14) 22
3.11 Critical Path and Performance Equation (L14) 22
3.12 Logic Faults and Stuck-at Errors (L14) 23

4 Pipelining and Hazards (Lectures 15–18) 23

4.1 Pipelining Basics: Throughput vs Latency (L15) 23
4.2 MIPS 5-Stage Pipeline and Diagram (L15) 24
4.3 Pipeline Terminology: Fill, Full, Drain, Balance (L15) 24
4.4 Pipeline Registers and Overhead (L15, L16) 25
4.5 Structural Hazards (L16) 25
4.6 Data Hazards: RAW, WAR, WAW (L16, L17) 26
4.7 Half-Cycle Register Write/Read Fix (L16) 26
4.8 Load-Use Hazards and Bubbles (L16, L17) 26
4.9 Forwarding (Bypassing) (L16, L17) 27
4.10 Software Interlocking and Scheduling (L17) 27
4.11 Control Hazards and Branch Penalty (L17, L18) 28

4.12 Early Branch Resolution (L18) . . . 28
4.13 Branch Prediction (L18) . . . 29
4.14 1-bit vs 2-bit Predictors (L18) . . . 29
4.15 Delay Slots (L18) . . . 29
4.16 Pipeline Performance with Stalls (L18) . . . 30

5 Memory Hierarchy, Caches, and Virtual Memory (Lectures 18–20) 30

5.1 Memory Hierarchy Levels and Tradeoffs (L18, L20) . . . 30
5.2 Principle of Locality (L18) . . . 31
5.3 Cache Basics: Hit, Miss, Block Size, I-Cache and D-Cache (L18) . . . 31
5.4 Direct-Mapped Cache Mapping, Tags, and Valid Bit (L18, L19) . . . 32
5.5 Direct-Mapped Cache Example (L18) . . . 32
5.6 Cache Miss Types (L19, L20) . . . 33
5.7 AMAT and AMSC Formulas (L19, L20) . . . 33
5.8 Cache Management Policies (L19, L20) . . . 33
5.9 Write-Through vs Write-Back and Dirty Bit (L19) . . . 34
5.10 Write-Allocate vs No-Write-Allocate (L19) . . . 34
5.11 Set-Associative and Fully Associative Caches (L19, L20) . . . 35
5.12 Replacement Policies (LRU, LFU, Random) (L19) . . . 35
5.13 Cache Performance Example (L19) . . . 36
5.14 Multiword Blocks, Critical Word First, Early Restart (L20) . . . 36
5.15 Cache Optimization Knobs (L20) . . . 36
5.16 Virtual Memory Basics (L20) . . . 37
5.17 Page Tables and PTE Fields (L20) . . . 37
5.18 Page Faults and Handler Steps (L20) . . . 38
5.19 Page Replacement and Reference Bits (L20) . . . 38
5.20 TLB and TLB Miss Handling (L20) . . . 39

A MIPS Programming Guide (Lecture Notes-Compatible) 39

A.1 Register File and Calling Convention . . . 39
A.2 Data Types, Directives, and Alignment . . . 40
A.3 Instruction Formats and Addressing Modes . . . 40
A.4 Arithmetic and Logical Operations . . . 40
A.5 Shifts and Bit Tricks . . . 41

A.6 Load/Store and Memory Access 41
A.7 Branches and Jumps 41
A.8 Functions and Stack Frames 42
A.9 Syscalls (MARS/QtSPIM) 42
A.10 Arrays and Loops 43
A.11 Strings 44
A.12 Multiply/Divide Instructions 44
A.13 Floating-Point Programming 44
A.14 Common Debugging Checklist 45

How to Use This Document to Pass the Exam

- The **Index** (Table of Contents) lists every concept with real page numbers after PDF compilation. Use it as your entry point.
- For each concept, read the **Step-by-step** list first, then the **Examples**. This mirrors how exam questions are structured.
- Use the **Common Checks and Traps** line to avoid typical mistakes under time pressure.
- If you are stuck, jump to the **MIPS Programming Guide** appendix for syntax, syscall, and pattern refreshers.
- When you get additional past papers, we can replace or tune examples to match difficulty and style.

Diagram Placeholders

These notes include placeholders for diagrams. Provide screenshots and place them in the `images/` folder as requested. I will wire them into the document.

1 Floating Point and Computer Arithmetic (Lectures 10–11)

1.1 IEEE 754 Formats and Fields (L10, L11)

What it is. Floating point numbers are stored as sign, exponent, and fraction. Single precision: 1 sign bit, 8 exponent bits, 23 fraction bits. Double precision: 1 sign bit, 11 exponent bits, 52 fraction bits. Normalized value:

$$\text{value} = (-1)^S \times (1.F) \times 2^{E-\text{bias}}$$

where bias is 127 (single) or 1023 (double).

Step-by-step.

1. Split the bits into S, E, and F.
2. Restore the hidden 1 for normalized values (E not all zeros or ones).
3. Compute $e = E - \text{bias}$.
4. Compute $(1.F)$ as a binary fraction and apply sign.

Examples.

1. Single: $S = 0, E = 127, F = 0$ gives $+1.0$.
2. Single: $S = 1, E = 128, F = 1000...$ gives $-1.5 \times 2^1 = -3.0$.
3. Double: $S = 0, E = 1023, F = 0$ gives $+1.0$.

Common checks and traps. Do not forget the hidden 1 for normalized values.

Diagram placeholder: IEEE 754 single and double field layout (L10)

1.2 Exponent Bias, Encoding, and Special Exponents (L10, L11)

What it is. Exponents are stored with a bias so they can be compared as unsigned. Exponent all-zeros and all-ones are reserved for special values.

Step-by-step.

1. Convert actual exponent e to stored exponent $E = e + \text{bias}$.
2. If $E = 0$, the number is zero or denormal.
3. If E is all ones, the number is infinity or NaN.

Examples.

1. Single: $e = -3 \Rightarrow E = 124$ (binary 01111100).

2. Single: $e = 0 \Rightarrow E = 127$ (binary 01111111).

3. Single: $E = 255$ implies infinity or NaN (check fraction).

Common checks and traps. Do not treat $E = 0$ as a normal exponent.

1.3 Normalized, Denormal, Zero, Infinity, and NaN (L10)

What it is. Normalized values use a hidden 1. Denormals use exponent $E = 0$ and hidden bit 0 for gradual underflow. Special values use E all ones.

Step-by-step.

1. Normalized: $1.F \times 2^{E-\text{bias}}$.

2. Denormal: $0.F \times 2^{1-\text{bias}}$ (single: 2^{-126}).

3. Zero: $E = 0$ and $F = 0$ (both +0 and -0).

4. Infinity: E all ones and $F = 0$.

5. NaN: E all ones and $F \neq 0$.

Examples.

1. Smallest positive normal (single): 1.0×2^{-126}.

2. Smallest positive denormal (single): 2^{-149}.

3. Overflow in a multiplication produces $+\infty$.

Common checks and traps. Denormals have lower precision and a hidden 0, not 1.

Diagram placeholder: Normalized and denormal range on real line (L10)

1.4 Encoding and Decoding Procedure (L11)

What it is. Steps to map between decimal values and IEEE 754 bit patterns.

Step-by-step (encode).

1. Determine sign S.

2. Convert magnitude to binary and normalize as $1.F \times 2^e$.

3. Compute $E = e + \text{bias}$ and fill fraction bits.

4. Round to fit the fraction width.

Step-by-step (decode).

1. Read S, E, F and detect special values.
2. Restore the hidden bit (if normalized).
3. Compute the final value using $(-1)^S(1.F)2^{E-\text{bias}}$.

Examples.

1. Encode -0.75: $-0.11_2 = -1.1_2 \times 2^{-1}$, $E = 126$, fraction 1000....
2. Decode 1 01111110 1100...: sign negative, $E = 126$, value -0.875.
3. Encode 6.5: $110.1_2 = 1.101_2 \times 2^2$, $E = 129$, fraction 1010....

Common checks and traps. Normalization changes exponent; do not drop it.

1.5 Floating-Point Addition and Subtraction Algorithm (L10, L11)

What it is. FP add/sub aligns exponents, adds significands, normalizes, and rounds. Signs are handled using 2's complement when needed.

Step-by-step.

1. Align binary points by shifting the smaller-exponent significand right.
2. Add or subtract significands (use 2's complement for opposite signs).
3. Normalize the result (shift left or right; adjust exponent).
4. Round to target precision; renormalize if rounding carries.
5. Check for overflow/underflow.

Examples.

1. $1.111_2 \times 2^{-1} + 1.011_2 \times 2^{-3}$: align, add, normalize, round.
2. $1.000_2 \times 2^{-3} + (-1.000_2) \times 2^2$: convert to 2's complement, add, normalize.
3. Opposite signs with close magnitudes: result needs leading-zero normalization.

Common checks and traps. Alignment can drop low bits; use guard/round/sticky.

1.6 Floating-Point Multiplication Algorithm (L10)

What it is. Multiply significands, add exponents (minus bias), compute sign, normalize, round, and check range.

Step-by-step.

1. Sign: $S = S_X \oplus S_Y$.

2. Exponent: $E_Z = E_X + E_Y - \text{bias}$.
3. Multiply significands (may be 2p bits).
4. Normalize (shift right at most 1; adjust exponent).
5. Round and check overflow/underflow.

Examples.

1. $(1.010)_2 \times 2^{-1} \times (-1.110)_2 \times 2^{-2}$: sign negative, exponent -3.
2. Multiply by 2^k: only exponent changes (fast path).
3. Product of two numbers just under 2.0 may require normalization shift.

Common checks and traps. The product can have 2 leading bits; normalize once.

1.7 Guard, Round, and Sticky Bits (L10)

What it is. Extra bits retained during alignment and arithmetic to improve rounding accuracy.

Step-by-step.

1. Guard bit: first bit beyond fraction.
2. Round bit: next bit after guard.
3. Sticky bit: OR of all remaining shifted-out bits.
4. Use these bits to decide round-to-nearest (or other mode).

Examples.

1. Guard=1, sticky=1 usually rounds up.
2. Guard=0, sticky=1 usually rounds down.
3. Guard=1, round=0, sticky=0 is a tie (round-to-even).

Common checks and traps. Sticky is OR of all discarded bits, not just one bit.

1.8 Floating-Point Division Overview (L11)

What it is. FP division computes A/B by forming the reciprocal of B's significand and multiplying.

Step-by-step.

1. Compute sign: $S = S_A \oplus S_B$.

2. Exponent: $E_C = E_A - E_B + \text{bias}$.
3. Compute reciprocal of significand $1/S_B$ using iteration.
4. Multiply S_A by $1/S_B$, normalize, and round.

Examples.

1. Divide by power of two: only exponent changes.
2. A/B with B in $[1, 2)$ fits the reciprocal iteration range.
3. A/B with $B = 1.0$ leaves significand unchanged.

Common checks and traps. No remainder in FP division; treat as real division.

1.9 Goldschmidt Division Method (L11)

What it is. Computes reciprocal using multiplications and additions to approximate $1/S_B$.

Step-by-step.

1. Let $S_B = 1 + X$ where $0 < X < 1$.
2. Transform to $1/(1 - Y)$ with $Y = X/2$ and iterate $(1 + Y)(1 + Y^2)(1 + Y^4)$....
3. Use FP multipliers to compute powers and accumulate.
4. Multiply by S_A and adjust exponent.

Examples.

1. $S_B = 1.11_2$ gives $X = 0.11_2$ and $Y = 0.001_2$.
2. Four multiplications and five additions give a usable reciprocal for single precision.
3. If Y is small, higher powers like Y^{32} become negligible.

Common checks and traps. Ensure S_B is normalized so $1 < S_B < 2$.

1.10 Newton-Raphson Division Method (L11)

What it is. Finds reciprocal using root finding on $f(x) = 1/x - b$ with fast convergence.

Step-by-step.

1. Choose x_0 from a lookup table, with $0.5 < x_0 \leq 1$.
2. Iterate $x_{i+1} = x_i(2 - bx_i)$.
3. Stop when precision matches target (e.g., 23 bits for single).

4. Multiply by S_A and normalize.

Examples.

1. $b = 1.5$, $x_0 = 0.75$ gives $x_1 = 0.75(2 - 1.125) = 0.65625$.
2. Error roughly squares each iteration; 5 iterations suffice for single precision.
3. If x_0 is poor, more iterations are needed.

Common checks and traps. Keep b in $(1, 2)$ by normalizing the input.

1.11 FP Hardware: Multi-Cycle and Pipelined Units (L10, L11)

What it is. FP adders and multipliers are more complex than integer units and usually take multiple cycles; they can be pipelined.

Step-by-step.

1. Compare exponents and align.
2. Perform significand operation.
3. Normalize and round.
4. Detect overflow/underflow.

Examples.

1. An FP adder may take several cycles; integer add is usually 1 cycle.
2. Pipelined FP units allow throughput ¿ 1 op per cycle.
3. A slow FP unit sets the clock period if it is on the critical path.

Common checks and traps. FP pipelines improve throughput, not single-op latency.

1.12 MIPS FPU Architecture (L10)

What it is. The FP unit is coprocessor 1 with its own 32 registers $f0$–$f31$.

Step-by-step.

1. Single precision uses one register.
2. Double precision uses even-odd pairs ($f0/f1$, $f2/f3$, ...).
3. FP ops only use FP registers.

Examples.

1. Use $f2$ for a single precision value and $f4/f5$ for a double.
2. $add.sf0, f1, f6$ is a single-precision add.
3. $mul.df4, f4, f6$ uses double-precision pairs.

Common checks and traps. Double precision must start at even register numbers.

1.13 MIPS FP Instructions and Data Movement (L10)

What it is. Separate FP arithmetic, load/store, move, convert, and compare instructions.

Step-by-step.

1. Use `add.s/sub.s/mul.s/div.s` for single.
2. Use `add.d/sub.d/mul.d/div.d` for double.
3. Use `lwc1/swc1` and `ldc1/sdc1` for loads/stores.
4. Use `mfc1/mtc1` to move between GPR and FPR.
5. Use `c.eq.s/c.lt.s/c.le.s` and `bc1t/bc1f` for FP branches.

Examples.

1. Load, add, store: `lwc1 f2, 0(t0); lwc1 f4, 4(t0); add.s f6, f2, f4; swc1 f6, 8(t0)`.
2. Convert int to float: `mtc1 t0, f0; cvt.s.w f2, f0`.
3. Compare and branch: `c.lt.s f2, f4; bc1t label`.

Common checks and traps. FP branches use the FP condition flag, not integer flags.

1.14 FP Syscalls and Example (L10)

What it is. MARS syscalls include read/print float and double, and FP arguments use $f12$.

Step-by-step.

1. Set $v0 = 2$ to print float, $v0 = 3$ for double.
2. Place value in $f12$ and call `syscall`.
3. For reading, use $v0 = 6$ (float) or $v0 = 7$ (double).

Examples.

1. Print float: `li v0,2; mov.s f12,f0; syscall`.
2. Read float: `li v0,6; syscall` (result in $f0$).
3. Fahrenheit to Celsius: $C = (5.0/9.0)(F - 32.0)$ using `sub.s, div.s, mul.s`.

Common checks and traps. $f12$ is the argument register for FP prints.

1.15 Integer Multiplication Complexity (L10)

What it is. Classical and faster multiplication algorithms have different asymptotic costs.

Step-by-step.

1. School method: $\Theta(n^2)$ bit operations.
2. Karatsuba: reduce 4 multiplies to 3, $O(n^{\log_2 3}) \approx O(n^{1.585})$.
3. Schonhage–Strassen: $O(n \log n \log \log n)$.

Examples.

1. For $n = 1024$, Karatsuba beats n^2 for large enough n.
2. Karatsuba splits 16-bit numbers into two 8-bit halves.
3. As n grows, $n \log n \log \log n$ becomes far smaller than n^2.

Common checks and traps. Asymptotic improvement appears only for large n.

2 Integer Arithmetic and ALU (Lectures 11–12)

2.1 Unsigned Shift-Add Multiplier (L12)

What it is. Hardware multiplication by repeated add and shift using a 64-bit product register.

Step-by-step.

1. Initialize product with multiplier in the low half, zeros in the high half.
2. If product LSB is 1, add multiplicand to the high half.
3. Shift the entire product right by 1.
4. Repeat for word width (e.g., 32 iterations).

Examples.

1. $0010_2 \times 0110_2 = 1100_2$ via four iterations.
2. $1000_2 \times 1001_2$ adds only on cycles where LSB is 1.
3. Multiply by 0 yields zero with no effective adds.

Common checks and traps. The add uses the high half of the product, not the low.

Diagram placeholder: Shift-add multiplier datapath (L12)

2.2 Signed Multiplication and Sign Handling (L12)

What it is. Signed product sign is XOR of operand signs; multiply magnitudes, then fix sign.

Step-by-step.

1. Compute sign of result: $S = S_A \oplus S_B$.
2. Convert operands to positive magnitudes.
3. Multiply as unsigned.
4. If $S = 1$, negate the product.

Examples.

1. $(-3) \times (+2) = -6$.
2. $(-3) \times (-2) = +6$.
3. $(-8) \times 0 = 0$ (sign irrelevant).

Common checks and traps. Ensure operands are converted using two's complement rules.

2.3 Booth's Encoding and Algorithm (L12)

What it is. Booth's algorithm reduces runs of 1s in the multiplier to fewer add/sub operations.

Step-by-step.

1. Look at current bit and the bit to the right (previous bit).
2. 10: subtract multiplicand; 01: add multiplicand; 00 or 11: no op.
3. Arithmetic shift right after each step.

Examples.

1. Multiplier 01111 gives $16x - x$ (one subtract, shifts).
2. Multiplier 1101 triggers a subtract at 10 and add at 01.
3. A run of 1s uses two operations, not one per bit.

Common checks and traps. Use arithmetic shift for signed values.

Diagram placeholder: Booth encoding table and example (L12)

2.4 Full Adder, Ripple-Carry Adder, and 2's Complement Subtraction (L11, L12)

What it is. A full adder computes sum and carry; ripple-carry chains adders; subtraction uses two's complement.

Step-by-step.

1. Full adder: Sum $= A \oplus B \oplus C_{in}$, $C_{out} = AB + BC_{in} + AC_{in}$.
2. Ripple-carry: chain n full adders; carry propagates left.
3. Subtraction: invert B and set $C_{in} = 1$ to add $A + (\sim B + 1)$.

Examples.

1. 1-bit: $A = 1, B = 1, C_{in} = 0$ gives Sum=0, Carry=1.
2. 4-bit RCA delay is about 4 full-adder delays.
3. $7 - 3$ in 4-bit two's complement: $0111 + 1101 = 0100$.

Common checks and traps. RCA delay is $O(n)$, which sets the clock.

2.5 Carry Lookahead Adder (CLA) (L11)

What it is. CLA uses generate/propagate to compute carries in parallel.

Step-by-step.

1. Generate: $g_i = a_i b_i$.
2. Propagate: $p_i = a_i + b_i$.
3. Carry: $c_{i+1} = g_i + p_i c_i$.
4. Expand for 4-bit group to compute $c_1..c_4$ directly.

Examples.

1. Compute $c_1 = g_0 + p_0 c_0$ for given bits.
2. For a 4-bit group, c_4 uses $g_3, p_3, g_2, ...$
3. CLA reduces delay compared to RCA at the cost of extra logic.

Common checks and traps. Use OR for propagate in these slides, not XOR.

2.6 Hybrid CLA/RCA Adders (L11)

What it is. Combine fast CLA within groups and ripple between groups to balance cost and speed.

Step-by-step.

1. Split the adder into groups (e.g., 4-bit groups).
2. Use CLA inside each group to compute local carries.
3. Ripple the group carries between groups.

Examples.

1. 12-bit adder as three 4-bit CLA groups.
2. Larger groups reduce ripple levels but increase logic cost.
3. Compare 12-bit pure RCA vs hybrid CLA+RCA delay.

Common checks and traps. Full CLA for large n has impractical cost.

2.7 32-bit MIPS ALU Design (L12)

What it is. A 32-bit ALU built from 1-bit slices supports AND, OR, ADD, SUB, and SLT.

Step-by-step.

1. Use control lines (Binvert, Op1, Op0) to select operation.
2. For SUB, invert B and set carry-in to 1.
3. For SLT, use the sign of $A - B$ and set bit 0.

Examples.

1. AND: Binvert=0, Op=00.
2. ADD: Binvert=0, Op=10.
3. SUB: Binvert=1, Op=10.

Common checks and traps. SLT is based on signed comparison of $A - B$.

2.8 Overflow Detection and SLT Support (L12)

What it is. Signed overflow in add/sub occurs when carry into MSB differs from carry out.

Step-by-step.

1. Compute carries into and out of bit 31.
2. Overflow = carry31 XOR carry32.
3. SLT uses the sign of the subtraction result with overflow adjustment.

Examples.

1. $0x7FFFFFFF + 1$ overflows to negative.
2. $0x80000000 - 1$ overflows to positive.
3. SLT sets result to 1 if $A < B$ else 0.

Common checks and traps. Overflow rules differ for signed vs unsigned adds.

2.9 MIPS Instruction Execution Cycle (L12)

What it is. Classic 5-stage flow: IF, ID/RR, EX, MEM, WB.

Step-by-step.

1. IF: fetch instruction and compute PC+4.
2. ID/RR: decode and read registers.
3. EX: ALU operation or address calculation.
4. MEM: data memory access for loads/stores.
5. WB: write result back to register file.

Examples.

1. R-type: uses IF, ID, EX, WB (MEM is idle).
2. lw: uses all 5 stages.
3. beq: uses IF, ID, EX (MEM/WB idle in single-cycle view).

Common checks and traps. Different instruction types still share the same stage order.

2.10 Combinational vs Sequential Elements and Register File (L12)

What it is. Combinational logic has no memory (ALU, mux). Sequential elements store state (registers).

Step-by-step.

1. Register file reads two registers and writes one per cycle.

2. Writes occur on the clock edge; reads are combinational.
3. Control signals gate the write operation.

Examples.

1. A register holds data across cycles using D flip-flops.
2. Read rs and rt while writing rd in the same cycle.
3. Write enable 0 leaves register unchanged.

Common checks and traps. Writes are edge-triggered; do not assume immediate updates mid-cycle.

3 Processor Design: Single-Cycle MIPS (Lectures 13–14)

3.1 ISA-Driven Datapath Design and Building Blocks (L13)

What it is. Start from ISA requirements and map to hardware blocks: ALU, registers, muxes, memory, adders.

Step-by-step.

1. Identify instruction types (R, load/store, branch, jump).
2. List required operations per instruction (read reg, ALU, memory, write reg).
3. Choose hardware blocks to cover all operations.
4. Connect blocks with multiplexers for shared paths.

Examples.

1. R-type needs register read, ALU, register write.
2. Load needs ALU for address and memory read.
3. Branch needs register compare and PC update.

Common checks and traps. Avoid assuming a block can do two functions in one cycle.

Diagram placeholder: Single-cycle datapath overview with control signals (L13–L14)

3.2 R-Type Datapath (L13)

What it is. Reg-reg instructions read two registers, compute in ALU, and write to rd.

Step-by-step.

1. Read rs and rt from register file.
2. Select ALU operation based on funct.
3. Write result to rd.

Examples.

1. add rd, rs, rt: RegDst=1, ALUSrc=0, RegWrite=1.
2. and rd, rs, rt: ALUOp selects AND.
3. slt rd, rs, rt: ALU computes set-less-than.

Common checks and traps. rd is destination for R-type, not rt.

3.3 Load Datapath (L13)

What it is. lw computes address in ALU, reads memory, writes to rt.

Step-by-step.

1. Read rs and sign-extend immediate.
2. ALU adds rs + immediate for address.
3. Read memory and write data to rt.

Examples.

1. lw rt, 16(rs): ALUSrc=1, MemRead=1, MemtoReg=1.
2. Base+offset address uses signed immediate.
3. lw uses RegDst=0 because rt is destination.

Common checks and traps. Immediate must be sign-extended for address calculation.

3.4 Store Datapath (L13)

What it is. sw computes address in ALU and writes register value to memory.

Step-by-step.

1. Read rs (base) and rt (data).

2. ALU adds rs + immediate for address.
3. Write rt data to memory.

Examples.

1. sw rt, 0(rs): MemWrite=1, RegWrite=0.
2. Store uses ALUSrc=1 to add offset.
3. sw does not write back to register file.

Common checks and traps. sw uses rt as source, not destination.

3.5 Branch Datapath (L13)

What it is. beq compares rs and rt and conditionally updates PC.

Step-by-step.

1. Read rs and rt.
2. ALU subtracts to set Zero flag.
3. Sign-extend and shift immediate left 2.
4. Add to PC+4 for target address.

Examples.

1. beq rs, rt, imm: Branch=1, ALUOp=sub.
2. If Zero=1, PCsrc selects branch target.
3. If Zero=0, PC continues with PC+4.

Common checks and traps. Branch offset is word-aligned, so shift left by 2.

3.6 Combining Datapaths with Multiplexers (L13)

What it is. Different instructions reuse hardware with muxes to select inputs and outputs.

Step-by-step.

1. Use ALUSrc mux to choose register vs immediate.
2. Use RegDst mux to choose rd vs rt.
3. Use MemtoReg mux to choose ALU vs memory data.
4. Use PCsrc mux for PC+4 vs branch target.

Examples.

1. lw uses ALUSrc=1 and MemtoReg=1.
2. R-type uses RegDst=1 and MemtoReg=0.
3. beq uses PCsrc=1 only if Zero=1.

Common checks and traps. Do not wire two sources together without a mux.

3.7 Control Signals for Single-Cycle MIPS (L14)

What it is. Control signals drive muxes and memory operations based on opcode.

Step-by-step.

1. Decode opcode to generate RegDst, ALUSrc, MemRead, MemWrite, MemtoReg, RegWrite, Branch, ALUOp.
2. Set control values for each instruction type.
3. Feed ALUOp to ALU control to select function.

Examples.

1. R-type: RegDst=1, ALUSrc=0, RegWrite=1, MemRead=0, MemWrite=0.
2. lw: RegDst=0, ALUSrc=1, MemRead=1, MemtoReg=1, RegWrite=1.
3. sw: ALUSrc=1, MemWrite=1, RegWrite=0.

Common checks and traps. Branch control also depends on Zero output.

3.8 ALU Control Logic (L14)

What it is. Combines ALUOp and funct to select ALU operation.

Step-by-step.

1. ALUOp=00 for load/store (add).
2. ALUOp=01 for beq (subtract).
3. ALUOp=10 for R-type (use funct).
4. Map funct to AND/OR/ADD/SUB/SLT codes.

Examples.

1. funct 100000 -¿ add.
2. funct 100010 -¿ subtract.
3. funct 101010 -¿ slt.

Common checks and traps. ALUOp only distinguishes classes; funct picks final op.

3.9 Main Control Unit Design (L14)

What it is. A combinational controller that outputs control signals from opcode bits.

Step-by-step.

1. Use opcode bits as inputs to logic.
2. Produce control signals for datapath.
3. Verify each instruction's control vector.

Examples.

1. For opcode 0 (R-type), RegDst=1 and ALUOp=10.
2. For opcode 35 (lw), MemRead=1 and MemtoReg=1.
3. For opcode 4 (beq), Branch=1 and ALUOp=01.

Common checks and traps. Single-cycle control is purely combinational.

3.10 Jump Instruction Handling (L14)

What it is. j uses a 26-bit target combined with upper PC bits.

Step-by-step.

1. Take upper 4 bits of PC+4.
2. Concatenate 26-bit target and 2 zeros (word aligned).
3. Select this as the next PC when jump opcode detected.

Examples.

1. Jump to 0x00401234 uses top 4 bits of PC+4.
2. j does not read registers.
3. j has a dedicated PC mux select line.

Common checks and traps. Jump address is pseudo-absolute within current 256MB region.

3.11 Critical Path and Performance Equation (L14)

What it is. Single-cycle CCT is set by the longest instruction path, typically load.

Step-by-step.

1. Find the longest datapath: IMem -¿ RegFile -¿ ALU -¿ DMem -¿ RegFile.

2. Set CCT to cover that path.
3. CPU time: IC $\times$ CPI $\times$ CCT.

Examples.

1. Load has the longest path, so it sets clock period.
2. Fast R-type instructions still wait for long CCT.
3. Pipelining lowers CCT at the cost of higher CPI.

Common checks and traps. Single-cycle design violates "make the common case fast".

3.12 Logic Faults and Stuck-at Errors (L14)

What it is. A signal line stuck at 0 or 1 can cause certain instructions to fail.

Step-by-step.

1. Identify which control or data line is stuck.
2. Determine which instructions depend on that line.
3. Evaluate functional impact (safe vs catastrophic).

Examples.

1. If ALUSrc stuck at 0, lw/sw will fail (need immediate).
2. If RegWrite stuck at 0, all reg writes fail.
3. If Branch control stuck at 1, PC updates incorrectly.

Common checks and traps. Some faults break only a subset of instructions.

4 Pipelining and Hazards (Lectures 15–18)

4.1 Pipelining Basics: Throughput vs Latency (L15)

What it is. Pipelining overlaps instruction stages to increase throughput without reducing single-instruction latency.

Step-by-step.

1. Split execution into stages (IF, ID, EX, MEM, WB).
2. Start a new instruction each cycle.
3. After pipeline fills, complete roughly one instruction per cycle.

Examples.

1. Laundry analogy: 4 loads complete faster with pipeline than sequential.
2. A 5-stage pipeline ideally gives near 5x throughput for long programs.
3. Latency of one instruction still spans 5 stages.

Common checks and traps. Pipelining boosts throughput, not single-op speed.

4.2 MIPS 5-Stage Pipeline and Diagram (L15)

What it is. MIPS pipeline stages: IF, ID/RR, EX, MEM, WB.

Step-by-step.

1. IF: fetch instruction and PC+4.
2. ID/RR: decode and read registers.
3. EX: ALU operation or address calculation.
4. MEM: access data memory.
5. WB: write result back.

Examples.

1. Pipeline diagram shows 5 instructions active at cycle 5.
2. R-type uses MEM as NOP stage for uniformity.
3. Load uses all stages; store uses MEM but no WB.

Common checks and traps. All instructions occupy the same number of stages (with NOPs).

Diagram placeholder: 5-stage pipeline timeline (L15)

4.3 Pipeline Terminology: Fill, Full, Drain, Balance (L15)

What it is. Pipeline fills, becomes full, then drains. Speedup is limited by the slowest stage.

Step-by-step.

1. Fill: initial cycles before all stages are active.
2. Full: steady state, one completion per cycle.
3. Drain: final cycles after last instruction enters.
4. Balance stages to minimize cycle time.

Examples.

1. 5-stage pipeline becomes full at cycle 5.
2. If EX is longest stage, CCT equals EX time.
3. Unbalanced stages reduce speedup below pipeline depth.

Common checks and traps. Speedup $< k$ even without hazards.

4.4 Pipeline Registers and Overhead (L15, L16)

What it is. Latches between stages hold intermediate values; they add overhead.

Step-by-step.

1. Insert registers between each pair of stages.
2. Propagate instruction bits and data each cycle.
3. Account for latch delay in CCT.

Examples.

1. IF/ID holds instruction and PC+4.
2. EX/MEM holds ALU result and control bits.
3. Latch overhead reduces ideal speedup.

Common checks and traps. Pipeline registers must carry control signals too.

4.5 Structural Hazards (L16)

What it is. Two stages need the same resource in the same cycle.

Step-by-step.

1. Identify shared resources (e.g., single memory).
2. If conflict, stall one instruction (bubble).
3. Prefer separate instruction and data memories.

Examples.

1. IF and MEM both need memory in the same cycle.
2. Separate I-cache and D-cache removes the conflict.
3. Multi-ported memory is another solution.

Common checks and traps. Structural hazards are solved by hardware duplication or stalls.

4.6 Data Hazards: RAW, WAR, WAW (L16, L17)

What it is. An instruction depends on data from a previous instruction that is not yet written.

Step-by-step.

1. RAW: read after write (most common).
2. WAW: write after write (ordering issue).
3. WAR: write after read (possible in out-of-order).

Examples.

1. RAW: `add $t1, ...` then sub uses `$t1`.
2. WAW: `lw` writes `$t1`, then `sub` writes `$t1` earlier.
3. WAR: out-of-order write overtakes read.

Common checks and traps. Classic in-order MIPS has RAW hazards; WAR/WAW are rare.

4.7 Half-Cycle Register Write/Read Fix (L16)

What it is. Write in first half-cycle, read in second half-cycle to reduce RAW stalls.

Step-by-step.

1. Write back at beginning of cycle.
2. Allow ID/RR to read updated value in same cycle.
3. This removes some, not all, RAW hazards.

Examples.

1. `add` writes `$t1` in WB half-cycle; next instruction reads in ID.
2. Removes one stall in some sequences.
3. Still fails for load-use when value not ready.

Common checks and traps. Load data arrives after MEM, so half-cycle does not fix load-use.

4.8 Load-Use Hazards and Bubbles (L16, L17)

What it is. A load followed immediately by a consumer needs a stall cycle.

Step-by-step.

1. Detect that the next instruction uses the load destination.

2. Insert a bubble (NOP) to delay the consumer.
3. Continue once data is available.

Examples.

1. `lw $t1, 0($t2)` then `add $t3, $t1, $t4` requires 1 stall.
2. With two consumers, multiple stalls may be needed.
3. Inserting a useful independent instruction can hide the stall.

Common checks and traps. Forwarding cannot bring data backward in time.

4.9 Forwarding (Bypassing) (L16, L17)

What it is. Use ALU or MEM results directly as ALU inputs for later instructions.

Step-by-step.

1. Add EX-¿EX and MEM-¿EX bypass paths.
2. Use forwarding control to select newest value.
3. Eliminate most RAW hazards.

Examples.

1. add then sub uses forwarded EX result.
2. MEM-¿EX forwards load data once in MEM/WB.
3. With forwarding, many sequences drop from 10 to 8 cycles.

Common checks and traps. Load-use still needs 1 stall even with forwarding.

Diagram placeholder: Forwarding paths and hazard detection (L17)

4.10 Software Interlocking and Scheduling (L17)

What it is. Compiler reorders instructions or inserts NOPs to avoid hazards.

Step-by-step.

1. Identify dependent instructions.
2. Move independent instructions into delay slots.
3. Insert NOPs when no safe instruction exists.

Examples.

1. Swap a load with a later independent add.
2. Insert a NOP after load when no independent work exists.
3. Reorder loop body to hide load latency.

Common checks and traps. Reordering must preserve program semantics.

4.11 Control Hazards and Branch Penalty (L17, L18)

What it is. The next PC is unknown until branch outcome is known.

Step-by-step.

1. In classic MIPS, branch target is resolved in MEM stage.
2. This causes a 3-cycle branch penalty.
3. Stall or flush instructions until outcome is known.

Examples.

1. beq followed by lw: lw may be squashed if branch taken.
2. 3 stalls per branch if no prediction is used.
3. Moving compare earlier reduces penalty to 1.

Common checks and traps. PC update point determines penalty length.

Diagram placeholder: Branch hazard pipeline timing (L18)

4.12 Early Branch Resolution (L18)

What it is. Move compare and target computation to ID/RR to reduce stalls.

Step-by-step.

1. Add a zero-checker in ID.
2. Compute branch target in ID.
3. Update PC earlier if branch is taken.

Examples.

1. Reduces branch penalty from 3 to 1 cycle.
2. Can create new hazards if branch depends on prior load.
3. Needs extra ALU hardware for ID stage.

Common checks and traps. Early branch logic can conflict with earlier instruction writes.

4.13 Branch Prediction (L18)

What it is. Predict branch outcome to avoid stalls; correct prediction avoids penalty.

Step-by-step.

1. Predict-not-taken: fetch PC+4 by default.
2. Static prediction: backward taken, forward not-taken.
3. Dynamic prediction: keep history per branch.

Examples.

1. 47% of MIPS branches are not taken on average.
2. A loop branch is usually taken; static predictor exploits this.
3. Dynamic prediction stores last outcome in a history table.

Common checks and traps. Wrong prediction requires pipeline flush.

4.14 1-bit vs 2-bit Predictors (L18)

What it is. 2-bit predictors reduce flip-flopping on loop exits.

Step-by-step.

1. 1-bit predictor flips on every misprediction.
2. 2-bit predictor changes prediction only after two misses.
3. Improves accuracy for loop branches.

Examples.

1. Inner loop exit causes two mispredictions with 1-bit predictor.
2. 2-bit predictor stays in "taken" state through one miss.
3. For random branches, 2-bit may not help much.

Common checks and traps. Prediction state must be updated on actual outcome.

4.15 Delay Slots (L18)

What it is. Compiler places a useful instruction after a branch to fill the delay slot.

Step-by-step.

1. Identify the branch delay slot.

2. Move a safe instruction into that slot.
3. If none exists, insert NOP.

Examples.

1. Fill with an instruction from fall-through path.
2. Fill with an instruction from taken path if safe.
3. Use NOP if reordering breaks correctness.

Common checks and traps. Delay-slot instruction always executes, taken or not.

4.16 Pipeline Performance with Stalls (L18)

What it is. Pipeline speedup drops with stall cycles.

Step-by-step.

1. Ideal CPI for pipeline is 1.
2. Actual CPI = 1 + avg stall cycles per instruction.
3. Speedup = pipe depth/CPI.

Examples.

1. With branch frequency 0.2 and penalty 1: CPI = 1 + 0.2.
2. With load-use stalls 0.1 per instruction: CPI = 1.1.
3. Speedup of 5-stage pipeline with CPI 1.25 is 4.0.

Common checks and traps. CPI uses average stalls, not worst-case.

5 Memory Hierarchy, Caches, and Virtual Memory (Lectures 18–20)

5.1 Memory Hierarchy Levels and Tradeoffs (L18, L20)

What it is. Small, fast memories are costly; large memories are slow. Hierarchy bridges the gap.

Step-by-step.

1. Registers and L1 cache are fastest and smallest.
2. Lower levels (L2, L3, DRAM, disk) are larger and slower.
3. Data moves up and down the hierarchy in blocks.

Examples.

1. L1 cache is tens of KB; DRAM is GBs.
2. Disk is orders of magnitude slower than DRAM.
3. Tertiary storage (tape/cloud) is largest and slowest.

Common checks and traps. Faster memory almost always costs more per bit.

Diagram placeholder: Memory hierarchy pyramid (L18)

5.2 Principle of Locality (L18)

What it is. Programs reuse data nearby in time (temporal) and space (spatial).

Step-by-step.

1. Temporal locality: recent data likely used again.
2. Spatial locality: nearby addresses likely accessed soon.
3. Caches exploit both by storing recent blocks.

Examples.

1. Loop counter reused each iteration (temporal).
2. Sequential array access (spatial).
3. Instruction stream has strong spatial locality.

Common checks and traps. Locality is empirical, not guaranteed.

5.3 Cache Basics: Hit, Miss, Block Size, I-Cache and D-Cache (L18)

What it is. Cache stores blocks from lower memory. Hits are fast; misses incur penalty.

Step-by-step.

1. CPU access checks cache first.
2. On hit: serve from cache.
3. On miss: fetch block from lower level.
4. I-cache holds instructions; D-cache holds data.

Examples.

1. Hit time 1 cycle, miss penalty 100 cycles.
2. A 16-word block fetches adjacent words.
3. Separate I-cache and D-cache avoid structural hazards.

Common checks and traps. Block size is in words/bytes, not bits.

5.4 Direct-Mapped Cache Mapping, Tags, and Valid Bit (L18, L19)

What it is. Each memory block maps to exactly one cache line, identified by index and tag.

Step-by-step.

1. Split address into tag, index, and offset.
2. Use index to select cache line.
3. Compare tag and valid bit to decide hit/miss.

Examples.

1. 5-bit address, 8-line cache: 3-bit index.
2. Valid bit starts at 0 to avoid garbage hits.
3. Tag mismatch causes conflict miss.

Common checks and traps. Index is typically low-order bits above offset.

Diagram placeholder: Direct-mapped address split and mapping (L18, L19)

5.5 Direct-Mapped Cache Example (L18)

What it is. Step-by-step trace of hits and misses for a direct-mapped cache.

Step-by-step.

1. Start with all valid bits 0.
2. Process each address; compute index and tag.
3. Update cache line on misses.

Examples.

1. Address sequence 22, 26, 22, 26, 16, 3, 16, 18.
2. First accesses are misses; repeats may hit if not evicted.
3. Conflicts occur when two blocks map to same index.

Common checks and traps. A hit requires both tag match and valid=1.

5.6 Cache Miss Types (L19, L20)

What it is. Misses are classified as compulsory, capacity, or conflict.

Step-by-step.

1. Compulsory: first time a block is accessed.
2. Capacity: cache cannot hold all working set blocks.
3. Conflict: direct-mapped collisions or low associativity.

Examples.

1. First reference in a program is compulsory.
2. Large array that exceeds cache size causes capacity misses.
3. Two hot arrays mapping to same index cause conflict misses.

Common checks and traps. Conflict misses can be reduced by higher associativity.

5.7 AMAT and AMSC Formulas (L19, L20)

What it is. Average Memory Access Time and memory stall cycles quantify cache impact.

Step-by-step.

1. AMAT = hit time + (miss rate x miss penalty).
2. AMSC = (memory requests per instruction) x miss rate x miss penalty.
3. CPU time = IC x (Ideal CPI + AMSC) x CCT.

Examples.

1. Hit 1 cycle, miss 20 cycles, miss rate 5%: AMAT = 2 cycles.
2. If 36% of instructions are loads/stores, D-cache miss adds 1.44 cycles.
3. CPI goes from 2 to 5.44 in the example from lecture.

Common checks and traps. Use fractions, not percentages, in formulas.

5.8 Cache Management Policies (L19, L20)

What it is. Placement, identification, replacement, and write strategy decisions.

Step-by-step.

1. Placement: direct-mapped, set-associative, fully associative.

2. Identification: tag + index + valid bit.
3. Replacement: LRU, LFU, random.
4. Writes: write-through or write-back.

Examples.

1. Direct-mapped has no replacement choice.
2. 2-way set-associative chooses one of 2 lines.
3. Write-back uses a dirty bit per line.

Common checks and traps. Write policies affect miss handling details.

5.9 Write-Through vs Write-Back and Dirty Bit (L19)

What it is. Write-through updates memory immediately; write-back delays until eviction.

Step-by-step.

1. Write-through: update cache and memory on each store.
2. Write-back: update cache and set dirty bit.
3. On eviction, if dirty=1, write line to memory.

Examples.

1. Write-through simplifies consistency but increases memory traffic.
2. Write-back reduces traffic but needs dirty bit.
3. A write miss in write-back may trigger write of dirty victim.

Common checks and traps. Write-back can cause a write on a read miss.

5.10 Write-Allocate vs No-Write-Allocate (L19)

What it is. On a write miss, decide whether to bring the block into cache.

Step-by-step.

1. Write-allocate: fetch block, then write into cache.
2. No-write-allocate: write directly to memory.
3. Often combined as write-back + write-allocate or write-through + no-write-allocate.

Examples.

1. Write-through + no-write-allocate avoids loading on write miss.
2. Write-back + write-allocate keeps block for future writes.
3. Write buffer allows CPU to continue while memory writes complete.

Common checks and traps. Use consistent write policy pairs.

5.11 Set-Associative and Fully Associative Caches (L19, L20)

What it is. Allow multiple possible cache lines per index to reduce conflicts.

Step-by-step.

1. 2-way or n-way set-associative: search all entries in a set.
2. Fully associative: any block can go anywhere.
3. More associativity reduces conflict misses but increases hit time.

Examples.

1. For 8 lines, direct-mapped = 1-way, 2-way has 4 sets.
2. Fully associative needs tag compare for all lines.
3. 2-way set-associative can reduce misses from 5 to 4 in example.

Common checks and traps. Higher associativity needs more comparators.

Diagram placeholder: Set-associative cache structure (L19)

5.12 Replacement Policies (LRU, LFU, Random) (L19)

What it is. Choose which line to evict on a miss when multiple choices exist.

Step-by-step.

1. LRU: evict the least recently used line.
2. LFU: evict the least frequently used line.
3. Random: evict a random line, often near-LRU performance.

Examples.

1. 2-way cache uses 1 bit to track LRU.
2. 4-way LRU needs more state and logic.
3. Random replacement is simplest for high associativity.

Common checks and traps. LRU becomes complex as associativity grows.

5.13 Cache Performance Example (L19)

What it is. Compute CPI impact from I-cache and D-cache misses.

Step-by-step.

1. Compute I-cache miss cycles: miss rate x penalty.
2. Compute D-cache miss cycles: load/store fraction x miss rate x penalty.
3. Add to ideal CPI.

Examples.

1. I-miss 2% with 100-cycle penalty adds 2 cycles.
2. D-miss 4% with 36% loads adds 1.44 cycles.
3. CPI = 2 + 2 + 1.44 = 5.44.

Common checks and traps. Separate I and D miss contributions.

5.14 Multiword Blocks, Critical Word First, Early Restart (L20)

What it is. Larger blocks exploit spatial locality but increase miss penalty; critical word first reduces effective penalty.

Step-by-step.

1. On miss, fetch an entire block of multiple words.
2. Deliver the requested word first (critical word first).
3. Resume CPU early while the rest of block arrives.

Examples.

1. 16-word block reduces compulsory misses for sequential access.
2. Early restart reduces perceived miss penalty.
3. Too-large blocks increase conflict and capacity misses.

Common checks and traps. Block size tradeoff depends on workload.

5.15 Cache Optimization Knobs (L20)

What it is. Change block size, capacity, associativity, or add levels to improve performance.

Step-by-step.

1. Increase block size to reduce compulsory misses.
2. Increase cache size to reduce capacity misses.
3. Increase associativity to reduce conflict misses.
4. Use multi-level caches or software reordering.

Examples.

1. Loop interchange improves spatial locality.
2. L2 cache reduces miss penalty for L1 misses.
3. Higher associativity increases hit time and power.

Common checks and traps. Improvements can trade one miss type for another.

5.16 Virtual Memory Basics (L20)

What it is. MM acts as a cache for disk; programs see a larger virtual address space.

Step-by-step.

1. CPU generates logical (virtual) addresses.
2. OS and hardware translate to physical addresses.
3. Pages are the VM blocks; page faults occur on misses.

Examples.

1. 32-bit virtual address may map to 30-bit physical address.
2. Each process gets a private virtual address space.
3. A page fault triggers disk access and OS handler.

Common checks and traps. Page faults are orders of magnitude slower than cache misses.

5.17 Page Tables and PTE Fields (L20)

What it is. Page tables map virtual page numbers to physical page numbers.

Step-by-step.

1. Split virtual address into page number and offset.
2. Use page number to index the page table.
3. PTE stores physical page number and status bits (valid, dirty, reference).

Examples.

1. 4 KB pages mean 12-bit offset.
2. Valid bit 0 indicates page not in memory.
3. Dirty bit set when a page is modified.

Common checks and traps. Page tables live in physical memory, not in cache by default.

5.18 Page Faults and Handler Steps (L20)

What it is. A page fault occurs when the needed page is not in memory.

Step-by-step.

1. Trap to OS with faulting address.
2. Find page on disk; choose victim page.
3. Write back victim if dirty; load new page.
4. Update page table and restart instruction.

Examples.

1. Page fault costs millions of cycles.
2. Dirty victim causes extra disk write.
3. Restart from the faulting instruction after load.

Common checks and traps. Page faults are handled by OS, not hardware alone.

5.19 Page Replacement and Reference Bits (L20)

What it is. OS selects a page to evict using algorithms like LRU.

Step-by-step.

1. Use reference (use) bit to approximate LRU.
2. Periodically clear reference bits.
3. Prefer evicting pages with reference bit 0.

Examples.

1. Clock algorithm uses reference bits in a ring.
2. Dirty page eviction requires write-back.
3. Clean page eviction avoids disk write.

Common checks and traps. True LRU is expensive to implement in OS.

5.20 TLB and TLB Miss Handling (L20)

What it is. TLB caches page table entries to speed address translation.

Step-by-step.

1. Check TLB for virtual page number.
2. On hit, get physical page number and proceed.
3. On miss, fetch PTE from memory or trigger page fault.
4. Update TLB and retry.

Examples.

1. TLB miss with page in memory: load PTE and retry.
2. TLB miss with page not in memory: page fault.
3. Software-managed TLB uses a special exception handler.

Common checks and traps. TLB miss handling can be hardware or software.

A MIPS Programming Guide (Lecture Notes-Compatible)

A.1 Register File and Calling Convention

Key registers.

- $\$zero = 0$; $\$at$ assembler temp; $\$v0$–$\$v1$ return values.
- $\$a0$–$\$a3$ arguments; $\$t0$–$\$t9$ temporaries (caller-saved).
- $\$s0$–$\$s7$ saved (callee-saved); $\$sp$ stack pointer; $\$fp$ frame pointer; $\$ra$ return address.

Example: caller vs callee saved.

```
# caller uses $t0, must save if needed
add $t0, $a0, $a1
sw  $t0, 0($sp)

# callee saves $s0
sw  $s0, 0($sp)
...
lw  $s0, 0($sp)
jr  $ra
```

A.2 Data Types, Directives, and Alignment

Common directives. `.word`, `.half`, `.byte`, `.asciiz`, `.space`.

Examples.

```
.data
arr:   .word 1, 2, 3, 4
msg:   .asciiz "hello"
buf:   .space 64

.text
la  $t0, arr   # address of array
lw  $t1, 0($t0)
```

Alignment tip. Word data should be 4-byte aligned for lw/sw.

A.3 Instruction Formats and Addressing Modes

Formats. R-type (op, rs, rt, rd, shamt, funct), I-type (op, rs, rt, imm), J-type (op, target).

Addressing modes.

- Register: `add $t0, $t1, $t2`
- Immediate: `addi $t0, $t1, 4`
- Base+offset: `lw $t0, 8($sp)`
- PC-relative branch: `beq $t0, $t1, label`

A.4 Arithmetic and Logical Operations

Examples.

```
add  $t0, $t1, $t2   # signed add
addu $t0, $t1, $t2   # unsigned add (no overflow trap)
sub  $t0, $t1, $t2
and  $t0, $t1, $t2
or   $t0, $t1, $t2
xor  $t0, $t1, $t2
slt  $t0, $t1, $t2   # $t0=1 if $t1<$t2 (signed)
sltu $t0, $t1, $t2   # unsigned compare
```

Example: absolute value.

```
abs:
  slt  $t0, $a0, $zero
```

```
  beq  $t0, $zero, done
  sub  $a0, $zero, $a0
done:
  jr   $ra
```

A.5 Shifts and Bit Tricks

Examples.

```
sll $t0, $t1, 2     # multiply by 4
srl $t0, $t1, 1     # logical shift right
sra $t0, $t1, 1     # arithmetic shift right

# extract low 8 bits
andi $t0, $t1, 0xFF
```

A.6 Load/Store and Memory Access

Examples.

```
lw  $t0, 0($sp)     # load word
sw  $t0, 4($sp)     # store word
lb  $t1, 0($t0)     # load byte (sign-extend)
sb  $t1, 1($t0)     # store byte
lh  $t2, 0($t0)     # load half
sh  $t2, 2($t0)     # store half
```

Alignment tip. Use lw/sw on word-aligned addresses only.

A.7 Branches and Jumps

Examples.

```
beq $t0, $t1, equal
bne $t0, $t1, noteq

loop:
  addi $t0, $t0, -1
  bne  $t0, $zero, loop

j   target
jal func       # jump and link
jr  $ra         # return
```

A.8 Functions and Stack Frames

Pattern.

```
func:
  addi $sp, $sp, -16
  sw   $ra, 12($sp)
  sw   $s0, 8($sp)
  move $s0, $a0
  ...
  lw   $s0, 8($sp)
  lw   $ra, 12($sp)
  addi $sp, $sp, 16
  jr   $ra
```

Example: factorial (recursive).

```
fact:
  addi $sp, $sp, -8
  sw   $ra, 4($sp)
  sw   $a0, 0($sp)
  ble  $a0, 1, base
  addi $a0, $a0, -1
  jal  fact
  lw   $a0, 0($sp)
  mul  $v0, $v0, $a0
  j    done
base:
  li   $v0, 1
done:
  lw   $ra, 4($sp)
  addi $sp, $sp, 8
  jr   $ra
```

A.9 Syscalls (MARS/QtSPIM)

Common services.

- 1: print int (`$a0`)
- 4: print string (`$a0`)
- 5: read int (returns in `$v0`)
- 10: exit
- 2: print float (`$f12`), 6: read float (returns in `$f0`)

Example: print integer and newline.

```
li   $v0, 1
move $a0, $t0
syscall

li   $v0, 11
li   $a0, 10
syscall
```

A.10 Arrays and Loops

Example: sum array of N words.

```
sum:
  move $t0, $a0    # base
  move $t1, $a1    # N
  li   $v0, 0
loop:
  beq  $t1, $zero, done
  lw   $t2, 0($t0)
  add  $v0, $v0, $t2
  addi $t0, $t0, 4
  addi $t1, $t1, -1
  j    loop
done:
  jr   $ra
```

Example: max in array.

```
max:
  lw   $v0, 0($a0)
  addi $a0, $a0, 4
  addi $a1, $a1, -1
loop:
  beq  $a1, $zero, done
  lw   $t0, 0($a0)
  slt  $t1, $v0, $t0
  beq  $t1, $zero, skip
  move $v0, $t0
skip:
  addi $a0, $a0, 4
  addi $a1, $a1, -1
  j    loop
done:
  jr   $ra
```

A.11 Strings

Example: strlen.

```
strlen:
  move $t0, $a0
  li   $v0, 0
loop:
  lb   $t1, 0($t0)
  beq  $t1, $zero, done
  addi $v0, $v0, 1
  addi $t0, $t0, 1
  j    loop
done:
  jr   $ra
```

A.12 Multiply/Divide Instructions

Examples.

```
mult $t0, $t1
mflo $t2         # low 32 bits of product
mfhi $t3         # high 32 bits of product

div  $t0, $t1
mflo $t2         # quotient
mfhi $t3         # remainder
```

A.13 Floating-Point Programming

Example: dot product of 2 floats.

```
dot:
  li   $t0, 0
  mov.s $f0, $f0   # sum in f0
loop:
  beq  $t0, $a2, done
  lwc1 $f2, 0($a0)
  lwc1 $f4, 0($a1)
  mul.s $f6, $f2, $f4
  add.s $f0, $f0, $f6
  addi $a0, $a0, 4
  addi $a1, $a1, 4
  addi $t0, $t0, 1
  j    loop
done:
  jr   $ra
```

Example: int to float conversion.

```
mtc1 $t0, $f0
cvt.s.w $f2, $f0
```

A.14 Common Debugging Checklist

- Check register lifetimes: did you overwrite a needed *s* register?
- Check stack alignment and saved *ra* for nested calls.
- Verify load/store offsets and alignment.
- Confirm branch logic and delay slots in your environment.
- For FP, use correct .s/.d instruction variants.